HO-01

How to analyze algo:

→ Sorting: bubble sort

→ searching: binary search

Hints:

- try using while loops (will make loop invariants easier)
- $\Theta(1)$ operations:
  - add/subtract/multiply/divide
  - assignment/set/get
  - return
  - other simple things
- read pg 5 & 6 before trying page 5

0. Alg BubbleSort⟨A⟩: helpful for proof

```
1   unsorted ← true                          t ← n-1
2   while (unsorted):   (unsorted=true)
3       unsorted ← false                     t
4       for ℓ ∈ [1.. t-1]:
5           if A[ℓ] > A[ℓ+1]:
6               tmp ← A[ℓ]
7               A[ℓ] ← A[ℓ+1]
8               A[ℓ+1] ← tmp
9               unsorted ← true
10          end if
11      end for                              t ← t-1
12  end while
13  return A
```

"in place sort"

BinaryS

Input: sor
Output: in

```
1: start.
2: if A
3:     e
4: elif
5:     B
6: elif
7:     B
8: else:
9:     x
```

proof

n-1

BinarySearch[A, x]

Input: sorted arr A; find num x
Output: index of x

```
1: start_index = ⌊len(A)/2⌋
2: if A[start_index] == x:
3:     end; return start_index
4: elif A[start_index] > x:
5:     BinarySearch[0 : start_index, x]
6: elif A[start_index] < x:
7:     BinarySearch[start_index : -1, x]
8: else:
9:     x is not in A; end
```

t ← t-1

Typo in 9/3 notes:

$c \geq 1$ in big-O def'n

(was written $c > 0$)

$f(n)$ is $O(g(n))$ means:

$\exists\ c \in \mathbb{R},\ c \geq 1,\ n_0 > 0$ such that

$$f(n) \leq c\,g(n)$$

$$\Leftrightarrow \frac{1}{c} f(n) \leq g(n)$$

Also: g is $\Omega(f(n))$.

$\Theta = O + \Omega$

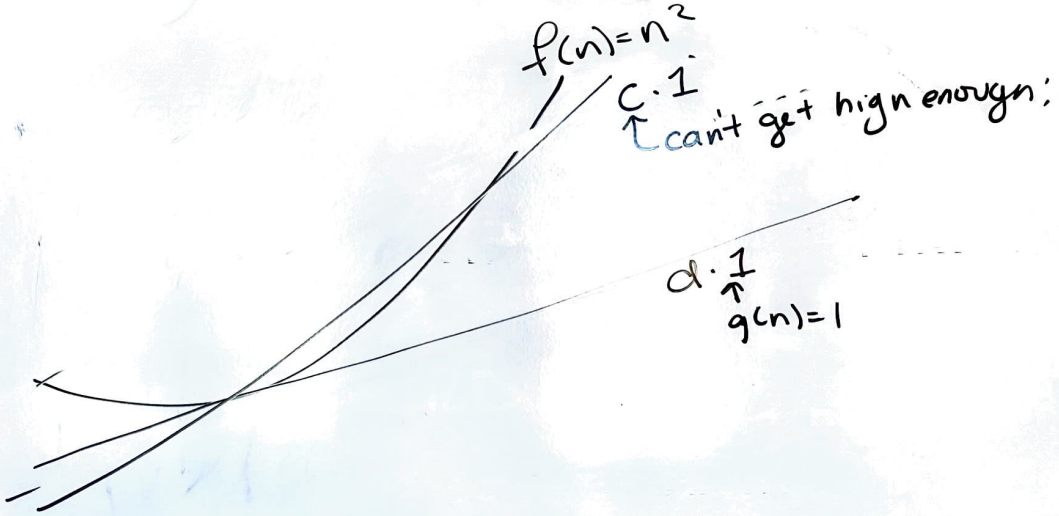
$f(n)$ is $\Theta(g(n))$ means:

$\exists\ c_1, c_2 \in \mathbb{R}$ s.t. $c_1, c_2 \geq 1$, $\forall n \geq n_0$ and $n_0 > 0$

$$\frac{1}{c_1} g(n) \leq f(n) \leq c_2 g(n)$$

Letting $c = \max c_1, c_2$

$$\frac{1}{c} g(n) \leq f(n) \leq c \cdot g(n)$$



$f(n)$ is: $\Omega(1)$

is not $O(1)$

$\therefore$ not $\Theta(1)$

LOOP INVARIANTS

G = loop guard
↳ = what keeps you in the loop
↳ easy for a while loop

Q = what is true at the end (= your goal!)

P = preconditions
= everything that is true before we enter the loop

L = what it needs to be to make I, M, & E work
(might take a few iterations to get it all)
↑ all statements = evaluated to T/F

f(n) can be

* worst-case runtime
  (my adversary is giving input)
  over all inputs of size n, what is the worst behavior.
  (usual analysis is here)

- best-case runtime
  IMO, is useless
  over all inputs of size n, what is the best runtime?

- average case
  (for randomized algos)